SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	ITA0312	**Course Title:**	Mobile computing
CO Assessed:	CO4	**Assessment:**	AT2 – CASE STUDY
Weightage:	30 % of CIA	**Total Marks:**	100
CO4 Statement:	*Identify the components and structure of mobile application for Android and Windows OS-based devices.*		
Student Name:	P.Ranjithkumar	**Reg. No.:**	192411119
Section / Batch:	2024	**Date:**	27.10.2026

Code of Conduct

I P.Ranjithkumar (192411119)____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P.Ranjithkumar

Instructions

- Duration: 60 minutes
- Number of questions : 01

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Introduction

A smartphone user reports that an application continues using location services even after its interface has been closed. Closing a screen does not always mean that the application process or its background work has stopped. The behaviour may be intentional, caused by a lifecycle or service-management issue, or indicate that the application is not respecting the user's expectations and privacy settings.

This assessment examines how application lifecycle management, background execution, operating-system location services, and power-management policies interact. The exact behaviour depends on the operating-system version, device manufacturer, permissions granted, and the way the application is implemented.

2. Application Lifecycle Management

A mobile application commonly moves between foreground, background, stopped, and terminated states. When a user closes an interface, the operating system may only hide or pause the activity rather than immediately terminate the whole application. Other components, such as a service, scheduled task, or location callback, may continue running.

For example, an app may register for location updates when it opens and fail to unregister the listener when the screen closes. The operating system can therefore continue delivering location updates to a component that remains active. A lifecycle bug can also occur when cleanup code is placed in the wrong callback or does not run on every exit path.

Correct handling requires the app to start location tracking only when needed and release location listeners, callbacks, and resources when tracking is no longer required. If tracking must continue after the interface closes, the app should make that purpose clear and follow the operating system's background-execution rules.

3. Background Execution and Services

Mobile operating systems provide controlled ways for apps to perform work while not visible. Depending on the platform and version, these can include foreground services, background services, scheduled work, tasks, or location-specific APIs. A navigation, fitness, safety, or delivery app may legitimately need location updates in the background.

A persistent service may continue after the user closes the app's main screen. On Android, a foreground service generally requires a visible notification and must satisfy applicable service-type and

permission requirements. Background location access may require separate permission and user approval. Exact restrictions differ between Android releases and device configurations.

If the app uses a permitted background mechanism, continued location use may be expected. If it starts background work without a valid need, fails to stop the service, or hides the activity from the user, the behaviour may be a design flaw or privacy concern.

4. Interaction with Operating-System Location Services

Location is usually provided through operating-system APIs that combine information from GPS, Wi-Fi, cellular networks, Bluetooth, and other available sources. The app typically requests updates from a system location service rather than controlling the hardware directly. The operating system checks the app's permissions and applies platform policies before providing location data.

Granting location permission does not necessarily mean that every use is appropriate. The app may have permission for background location, or it may receive location while a foreground service remains active. Conversely, if permission is revoked or restricted, the operating system should limit or deny further access according to the platform's rules.

The key distinction is between permission and purpose: permission makes access technically possible, but the app should use location only for a disclosed and necessary function. Users should be able to understand when tracking is active and how to stop it.

5. Power Management and Battery Impact

Location requests can increase battery consumption, particularly when an app requests frequent, high-accuracy GPS updates or keeps other sensors and network connections active. Operating systems use power-management features such as background limits, batching, deferred work, and battery-optimization policies to reduce unnecessary activity.

These controls may delay or limit background work, but they do not always immediately stop a permitted foreground service or all location activity. Manufacturer-specific battery managers may also behave differently. As a result, a location indicator or battery usage can remain visible even when the app's interface is closed.

A well-designed app should request an appropriate update frequency and accuracy, use passive or batched updates where suitable, and stop tracking as soon as the user-facing task ends. Power management is a safeguard, not a substitute for correct application lifecycle handling.

6. Possible Causes

Possible cause	How it can produce the behaviour	What to check
Lifecycle cleanup failure	A location listener or callback is not removed when the screen closes.	Review lifecycle callbacks and confirm that tracking is stopped on the intended exit path.
Background service remains active	A service continues location tracking independently of the interface.	Check active services, notifications, and the app's declared background components.
Background location permission	The app has permission to access location while not visible.	Review location permission settings and whether background access is necessary.
Scheduled or deferred work	A worker or task periodically requests location or restarts work.	Inspect scheduled jobs, task queues, and restart conditions.
OS or manufacturer policy	Power-management rules delay or limit work without immediately ending every component.	Compare behaviour with battery settings and the device's OS version.
Unclear or excessive tracking	The app continues collecting location beyond the function the user expects.	Compare actual behaviour with the app's disclosure and intended feature.

7. Diagnostic and Testing Plan

Test	Procedure	Expected result
Permission review	Inspect foreground and background location permissions.	Only permissions needed for the feature are granted.
Lifecycle test	Start tracking, close the	Tracking stops unless the user

	interface, then observe whether tracking stops.	has explicitly started a valid background task.
Service inspection	Check notifications and active/background services using permitted device tools.	Any continuing tracking is attributable to a clear, authorized component.
Location indicator test	Observe the OS location indicator and privacy dashboard where available.	Location access matches the app's stated behaviour.
Battery comparison	Compare battery use with tracking enabled and disabled over similar periods.	Unnecessary background activity is not sustained.
Permission revocation	Revoke location permission and repeat the test.	The app respects the changed permission and handles denied access safely.

8. Recommendations

1. Correct lifecycle handling: remove location updates and release resources when the tracking task ends.

2. Make background tracking explicit: explain why it is needed, when it operates, and how the user can stop it.

3. Apply least privilege: request foreground location first where suitable, and request background access only when the feature genuinely requires it.

4. Use approved background mechanisms: follow the target operating system's rules for foreground services, notifications, scheduled work, and location APIs.

5. Reduce power use: select suitable accuracy and update intervals, batch updates where possible, and avoid continuous GPS polling when not necessary.

6. Test across versions and devices: record the OS version, permission state, service state, battery settings, and observed result. Behaviour should not be assumed identical across all smartphones.

7. Protect privacy: do not collect, retain, or transmit location data beyond the stated purpose, and provide a clear way to disable tracking.

9. Conclusion

An application can continue using location after its interface is closed because the interface lifecycle is separate from the lifecycle of services and background tasks. If location listeners remain registered or a permitted background service continues running, the operating system may keep providing location data. Permissions and OS location services determine whether access is allowed, while power-management policies attempt to limit battery use without necessarily terminating every valid background task.

The behaviour should be investigated by checking lifecycle cleanup, active services, permissions, scheduled work, location indicators, and battery usage. The most reliable solution is correct application design combined with transparent user controls, least-privilege permissions, and compliance with the relevant operating-system version.

11. System Architecture Diagram

The architecture below shows how the interface, lifecycle handling, background components, operating-system permission checks, location sources, and power policies interact. The interface is only one part of the application; closing it does not necessarily stop a separate background component.

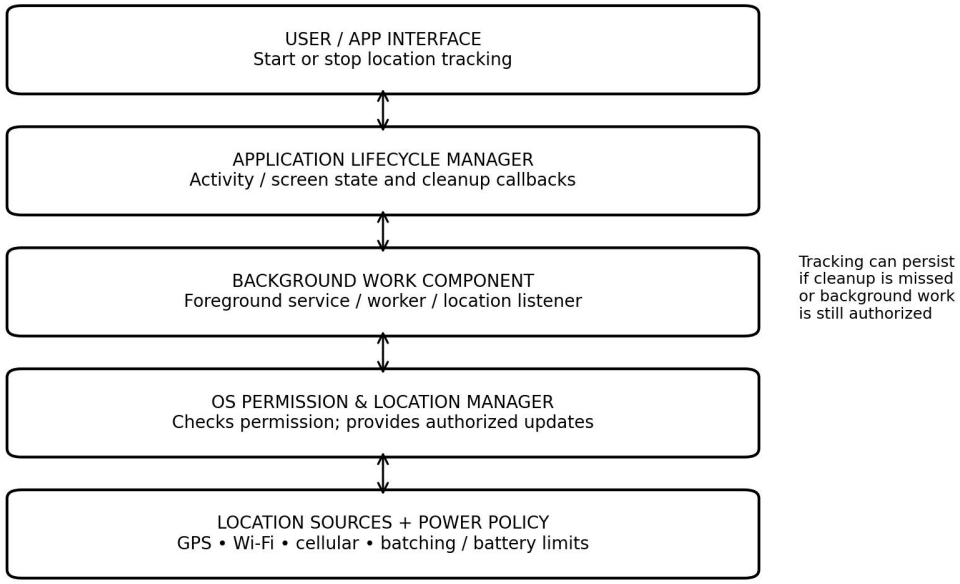


Figure 1. High-level architecture of background location access.

12. Application Lifecycle Flow Diagram

This flow shows why the same user action—closing the interface—can produce different results. If the app has correctly stopped its listener, tracking ends. If a permitted service or listener remains active, updates may continue until the task is stopped or the operating system restricts access.

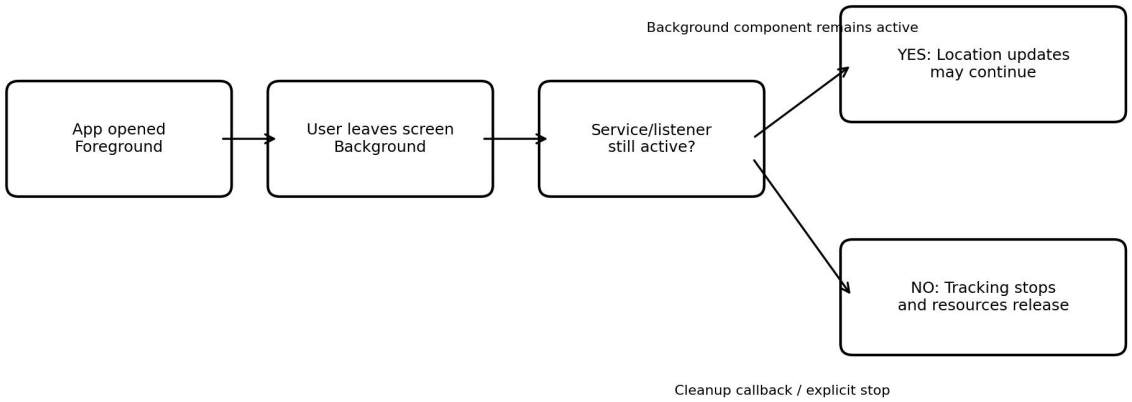


Figure 2. Lifecycle flow and the two possible outcomes after closing the interface.

13. Sequence Diagram: App-to-OS Location Interaction

The sequence diagram illustrates the order of events: the user starts tracking, the app requests location through the OS, and the OS returns updates if access is authorized. When the interface closes, the app should explicitly stop updates unless an ongoing background feature is intended and permitted.

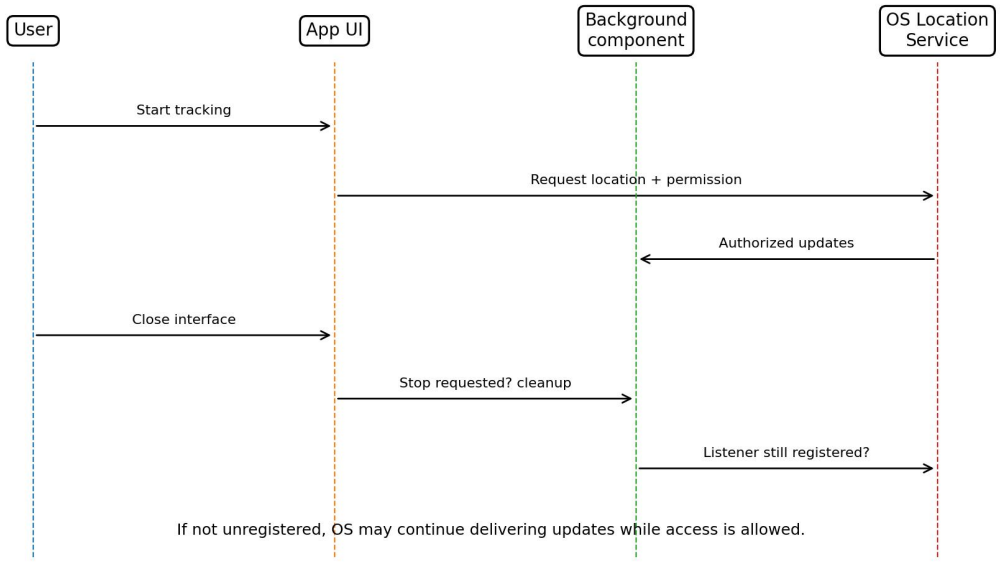


Figure 3. Simplified sequence of location requests and cleanup.

14. Diagnostic Flowchart

A systematic diagnosis should start with the location indicator and permission status, then inspect background services, lifecycle cleanup, and scheduled work. Repeat the test after stopping tracking and changing permission settings to determine whether the behaviour is expected or a defect.

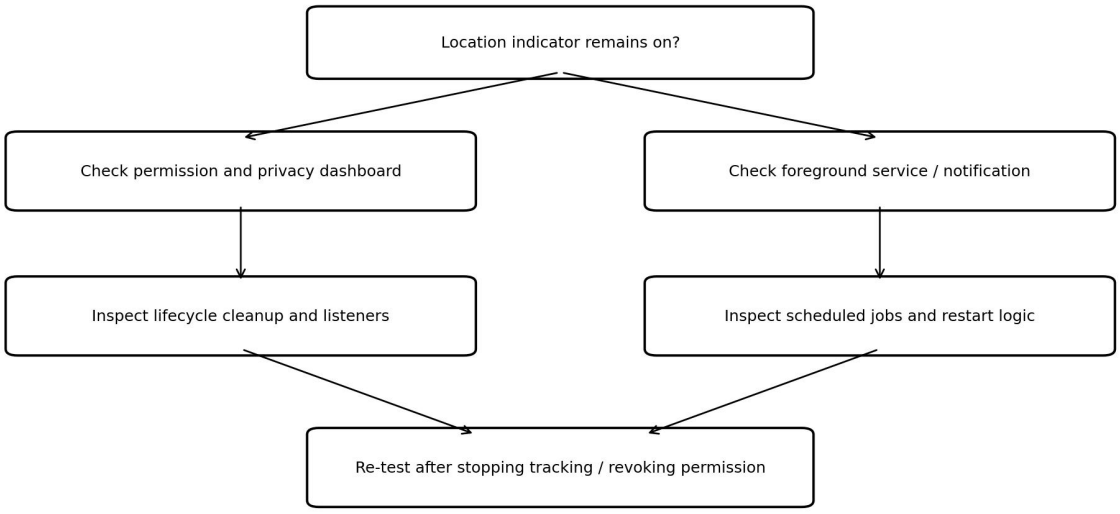


Figure 4. Diagnostic flow for persistent location access.

15. Component-Level Architecture and Responsibilities

Component	Main responsibility	Potential failure
User interface / Activity	Starts tracking, explains status, offers a stop control.	Closing the screen is incorrectly treated as a guaranteed stop.
Lifecycle controller	Starts and releases UI-bound resources at appropriate lifecycle events.	Cleanup is missed or occurs only on one exit path.
Location listener / callback	Receives location updates from the system API.	Listener remains registered after tracking should end.
Background service / worker	Runs approved work when the UI is not visible.	Work continues without a clear need or user-visible status.
Permission manager	Enforces location permission and background-access rules.	User grants broad access that is not needed for the feature.
Power manager	Limits background activity and helps reduce energy use.	Developer assumes battery controls will always terminate tracking.

16. Risk and Impact Analysis

Risk	Impact	Mitigation
Battery drain	Frequent GPS requests increase energy use.	Reduce update frequency and stop unnecessary requests.
Privacy exposure	Location may be collected beyond the user's expectation.	Use least privilege, clear disclosures, and a stop control.
Unexpected data transmission	Background location may be sent to a server without a clear user task.	Review data flows, retention, and purpose limitation.
Poor app reliability	Unnecessary services can consume resources or conflict	Use platform-approved background APIs and test

	with OS limits.	across OS versions.

17. Expected Results

A correct implementation should stop location updates when the relevant task ends, unless the user has knowingly started a feature that requires background tracking. The operating system should enforce permission rules, provide relevant privacy indicators, and apply background and power limits according to the device and OS version. Tests should record both expected and observed behaviour rather than assuming that closing the interface always terminates the process.